Appendix C

User Manual

Project: Car License Plate Recognition Smart Parking System
Student Name: Li Wenhao **Student ID:** 50087574

This User Manual provides clear, step-by-step instructions for installing, launching, operating, and evaluating the Car License Plate Recognition Smart Parking System. This manual is intended for general users and system testers with basic computer operation skills; no professional technical background is required. Following this guide, any user can independently run the system, perform license plate recognition, manage vehicle entry/exit processes, test system functions, and verify performance. This document is submitted as a formal appendix to the JC4000 Final Year Dissertation.

C.1 Prerequisites (System Requirements)

Ensure the following environment is prepared on a Windows computer:

- Python 3.10–3.11
- Node.js 16.x–18.x
- Chrome or Firefox browser
- Minimum 8 GB RAM
- 10 GB available storage space
- Tested Platform: Windows 10/11 64-bit

C.2 System Startup

Two startup methods are provided for flexible use.

C.2.1 One-Click Startup (Recommended)

1. Open the project root folder.
2. Double-click the script file `first_start.bat` (see Figure C.1).

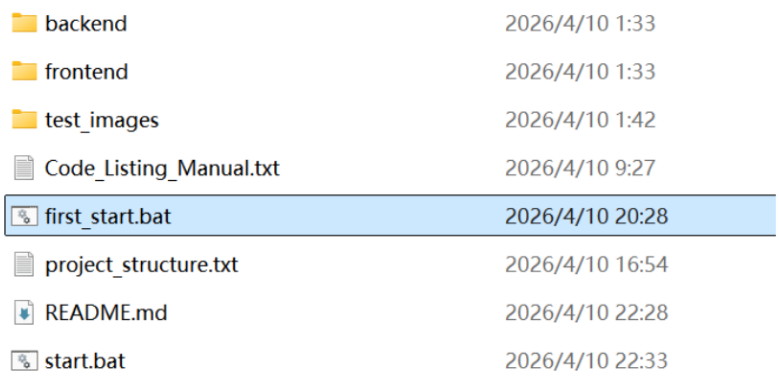


Figure C.1: Project root directory showing `first_start.bat`.

The script automatically completes:

- Backend dependency installation
- Frontend dependency installation
- Backend server startup (port 8000)
- Frontend server startup (port 3001)

Keep both terminal windows open during system operation.

C.2.2 Manual Startup

Backend

1. Open a terminal in the `backend` directory (see Figure C.2).

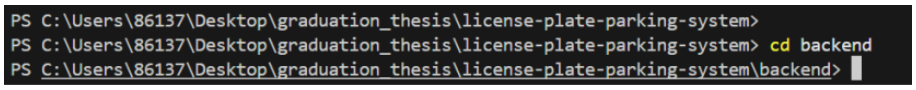


Figure C.2: Navigating to the backend directory in the terminal.

2. Install dependencies:

```
pip install -r requirements.txt
```

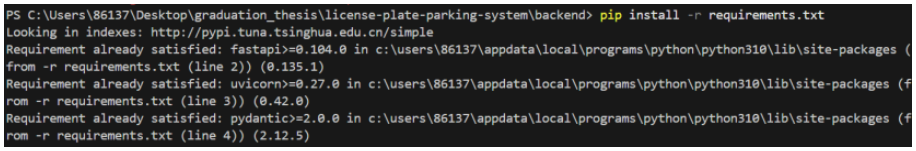


Figure C.3: Running `pip install -r requirements.txt`.

3. Start the service:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

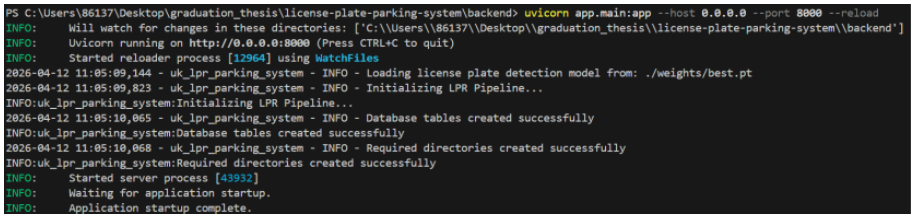


Figure C.4: Backend service startup output confirming successful initialisation.

Frontend

1. Open a new terminal in the `frontend` directory (see Figure C.5).

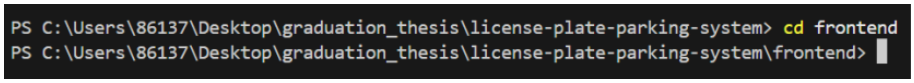


Figure C.5: Navigating to the frontend directory in the terminal.

2. Install dependencies:

```
npm install
```

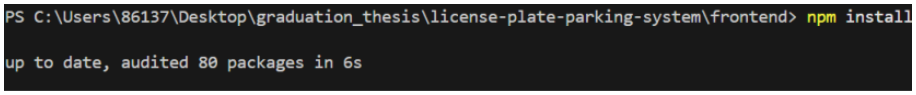


Figure C.6: Running `npm install` to install frontend dependencies.

3. Start the interface:

```
npm run dev
```



Figure C.7: Frontend development server started successfully on port 3001.

C.2.3 Access Addresses

- Frontend Interface: `http://localhost:3001` (see Figure C.8)
- Backend API Documentation: `http://localhost:8000/docs` or `http://localhost:8000/redoc` (see Figure C.9)

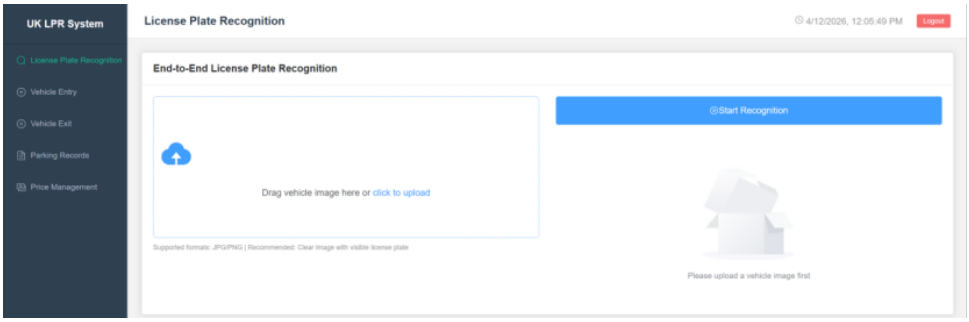


Figure C.8: System frontend interface at `http://localhost:3001`.

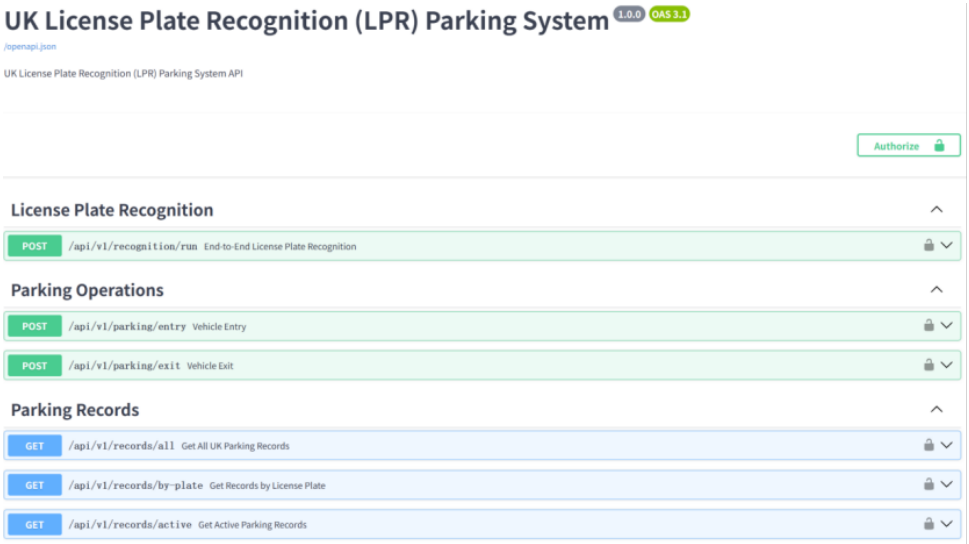


Figure C.9: Backend API documentation interface at `http://localhost:8000/docs`.

C.2.4 Input and Output Description

Input:

- Vehicle image files (JPEG/PNG format)
- Real-world vehicle images (normal, tilted, low-light, etc.)

Output:

- Recognised license plate number
- Accurate entry/exit timestamp
- Calculated parking duration
- Final computed parking fee

C.3 System Operation (Step-by-Step)

C.3.1 Administrator Login

1. Visit `http://localhost:3001` in a browser.

2. Enter the administrator account credentials.

3. Click **Login** to enter the main dashboard (see Figure C.10).

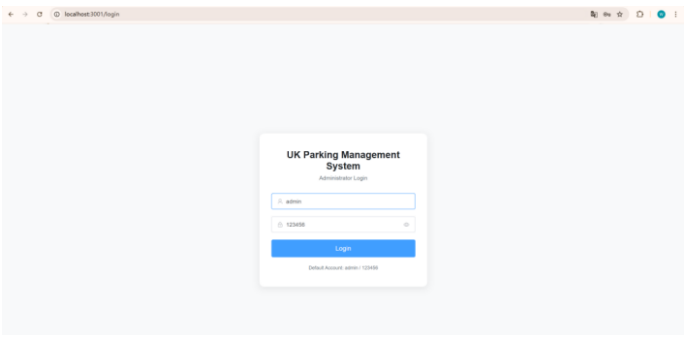


Figure C.10: Administrator login page.

C.3.2 License Plate Image Upload and Recognition

1. Navigate to the Recognition page.

2. Click **Upload Image** and select a test image from the `test_images` folder.

The system automatically executes:

- Image enhancement
- License plate detection
- Tilted plate perspective correction
- OCR character recognition
- License plate format validation

The final recognised plate number is displayed (see Figure C.11).

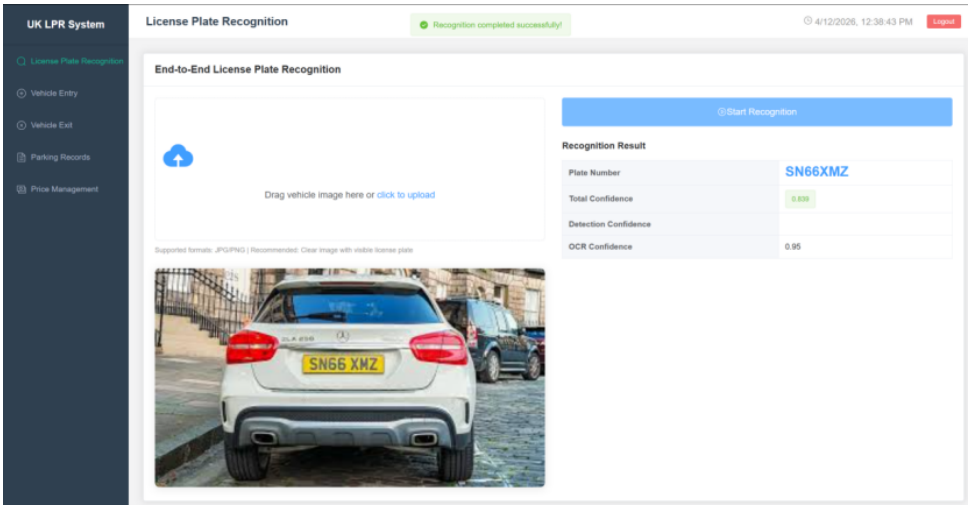


Figure C.11: License plate recognition result displayed on the Recognition page.

C.3.3 Vehicle Entry Registration

1. Confirm the recognised license plate number.
2. Click **Confirm Entry** to record the vehicle.

The system saves the plate number, entry time, and parking status (see Figure C.12).

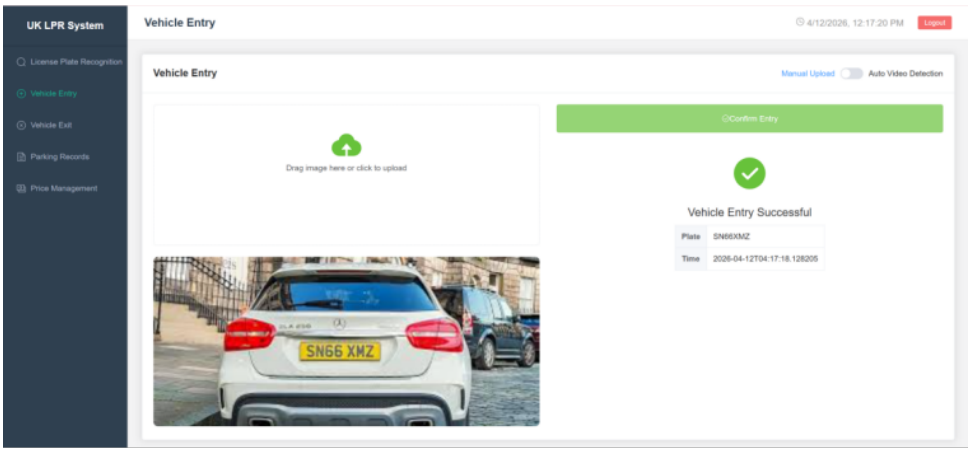


Figure C.12: Vehicle entry registration confirmation screen.

C.3.4 Vehicle Exit and Fee Calculation

1. Go to the Exit page.
2. Re-recognise or re-enter the license plate number.

The system automatically:

- Calculates total parking duration
- Computes parking fee based on configured rules
- Updates the vehicle status to Exited

See Figure C.13 for the exit confirmation screen.

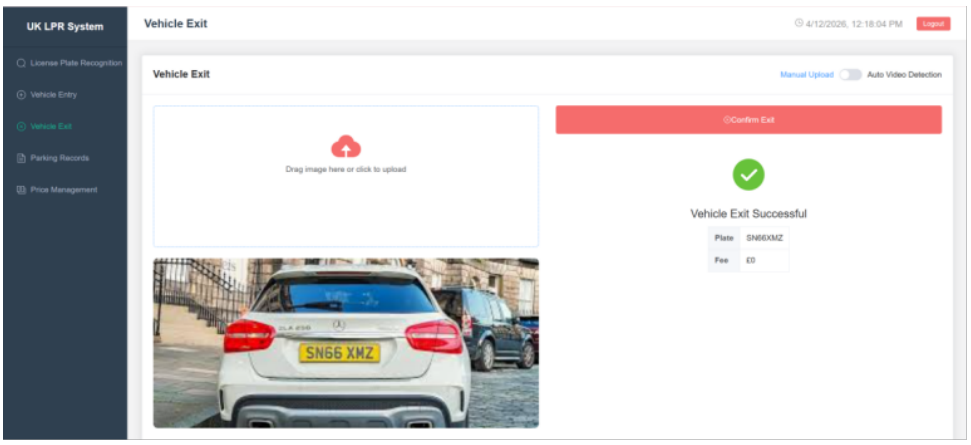


Figure C.13: Vehicle exit and fee calculation confirmation screen.

C.3.5 Parking Record Management

Open the Records page to view all historical parking data. Search, filter, and inspect details including time, duration, and fee (see Figure C.14).

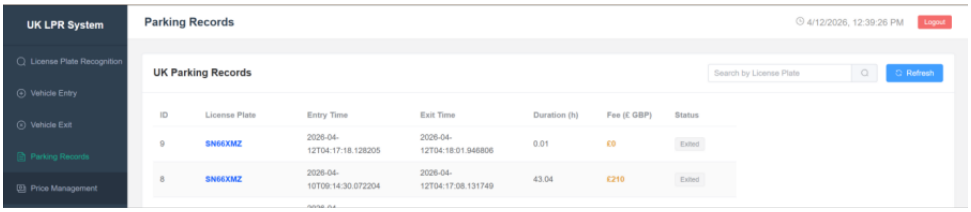


Figure C.14: Parking Records page showing historical entry/exit data.

C.3.6 Pricing Configuration

1. Access the Price Management page.
2. Modify free parking duration and hourly rate.
3. Click **Save** to apply new pricing rules.

See Figure C.15 for the pricing configuration interface.

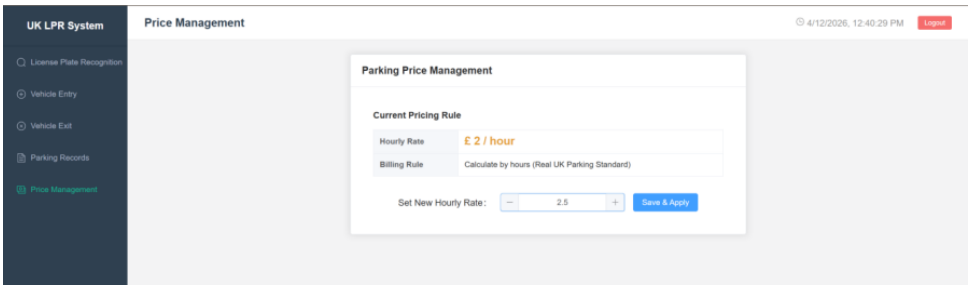


Figure C.15: Price Management page for configuring parking hourly rates.

C.4 System Performance Test

To formally evaluate system performance and robustness, test under the following conditions:

- Standard clear conditions
- Tilted/perspective-distorted plates
- Low-light environments

Perform repeated test runs (at least 10 iterations) and verify and record:

- License plate detection accuracy
- OCR recognition correctness
- System response speed and stability
- Reliability of entry/exit logic
- Accuracy of parking duration and fee calculation

C.5 Troubleshooting

C.5.1 Port 8000 / 3001 Occupied

- Close applications using the ports.
- Restart the system using the startup script.

C.5.2 Dependency Installation Failure

- Check network connectivity.
- Verify Python and Node.js are correctly installed and added to the system PATH.

C.5.3 Frontend Cannot Connect to Backend

- Ensure both server terminals are running.
- Check firewall and network settings.

C.6 Important Notes

- Do **NOT** close backend/frontend terminals during system use.
- The SQLite database is automatically created on first launch.
- All uploaded images are stored temporarily for verification.
- The system runs on CPU by default; GPU acceleration can be enabled by setting `DEVICE=cuda` in the configuration file if a compatible GPU is available.
- Test images are provided in the `test_images` folder for evaluation.